

# Tema 8 - Operaciones de entrada-salida

## Laboratorio Guiado

|                           |                                                                                                                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objetivos</b>          | Practicar operaciones de entrada-salida en Python: rutas con <code>pathlib</code> , lectura y escritura de ficheros de texto, CSV, JSON, <code>pickle</code> , ficheros binarios y conversión de datos CSV a JSON con validación. |
| <b>Material necesario</b> | Terminal, Python 3.13, venv del Tema08, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en <code>~/Curso_Python/Tema08</code> , y ficheros de datos en <code>Tema08/data</code> .                |

## Laboratorio 1. Operaciones previas con `pathlib.Path`

1. Actualizar el repositorio, abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
cd ~/Curso_Python
git pull origin main
source .venv/bin/activate
python --version
code .
```

2. Abrir el fichero `Tema08/src/01_pathlib_rutas.py`. Revisar la creación de rutas con `Path` y la diferencia entre ruta relativa y ruta absoluta.

```
from pathlib import Path

print("=== 1. Crear rutas con Path ===")
carpeta = Path("Tema08/data")
ruta = carpeta / "pathlib_demo.txt"
print("Carpeta relativa:", carpeta)
print("Ruta relativa:", ruta)
print("Ruta absoluta:", ruta.resolve())
```

3. Crear el directorio de trabajo si no existe y comprobar el tipo de recurso con `exists()` e `is_dir()`.

```
print("\n=== 2. Crear directorio si no existe ===")
carpeta.mkdir(parents=True, exist_ok=True)
print("Carpeta existe:", carpeta.exists())
print("Carpeta es directorio:", carpeta.is_dir())
print("Fichero existe antes de escribir:", ruta.exists())
```

4. Escribir y leer texto directamente con métodos de `Path`, sin usar `open()` explícitamente.

```
print("\n=== 3. Escribir y leer texto con métodos de Path ===")
ruta.write_text("ssh:22\nnginx:443\n", encoding="utf-8")
print("Fichero existe después de escribir:", ruta.exists())
print("Es fichero:", ruta.is_file())
print("Extensión:", ruta.suffix)
print("Contenido:")
print(ruta.read_text(encoding="utf-8").strip())
```

**5. Cambiar el nombre del fichero con `with_name()` y `rename()`.**

```
print("\n=== 4. Cambiar nombre con with_name() y rename() ===")
nueva_ruta = ruta.with_name("pathlib_demo_renamed.txt")
if nueva_ruta.exists():
    nueva_ruta.unlink()
ruta.rename(nueva_ruta)
print("Ruta original existe:", ruta.exists())
print("Ruta renombrada existe:", nueva_ruta.exists())
```

**6. Eliminar el fichero temporal con `unlink()`.**

```
print("\n=== 5. Limpiar fichero temporal ===")
if nueva_ruta.exists():
    nueva_ruta.unlink()
print("Ruta renombrada existe tras unlink:", nueva_ruta.exists())
```

**7. Listar ficheros CSV del directorio data con `glob()`.**

```
print("\n=== 6. Listar ficheros CSV del directorio data ===")
for fichero in carpeta.glob("*.csv"):
    print("CSV encontrado:", fichero.name)
```

**8. Ejecutar el script completo desde la terminal.**

```
python Tema08/src/01 pathlib rutas.py
```

## Laboratorio 2. Ficheros de texto

**1. Abrir el fichero `Tema08/src/02_ficheros_texto.py`. Revisar la apertura y cierre explícitos con `open()` y `close()`.**

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(parents=True, exist_ok=True)

print("=== 1. open() y close() explícitos ===")
ruta = carpeta / "texto_open_close.txt"
fichero = open(ruta, "w", encoding="utf-8")
fichero.write("Linea 1\n")
fichero.write("Linea 2\n")
fichero.write("Linea 3\n")
fichero.close()

fichero = open(ruta, "r", encoding="utf-8")
todo_el_texto = fichero.read()
fichero.close()
print(todo_el_texto.strip())
```

**2. Escribir un fichero de texto usando with para que Python cierre el recurso automáticamente.**

```
print("\n=== 2. Escritura recomendada con with ===")
ruta_servicios = carpeta / "servicios_texto.txt"
with open(ruta_servicios, "w", encoding="utf-8") as fichero:
    fichero.write(f"{'Servicio':<12} {'Puerto':>6} {'Estado':>8}\n")
    fichero.write(f"{'ssh':<12} {'22':>6} {'OK':>8}\n")
    fichero.write(f"{'https':<12} {'443':>6} {'OK':>8}\n")
    fichero.write(f"{'http':<12} {'80':>6} {'NA':>8}\n")
```

**3. Añadir nuevas líneas con el modo a sin sobrescribir el contenido anterior.**

```
print("\n=== 3. Añadir contenido con modo a ===")
with open(ruta_servicios, "a", encoding="utf-8") as fichero:
    fichero.write(f"{'ntp':<12} {'123':>6} {'OK':>8}\n")
    fichero.write(f"{'dns':<12} {'53':>6} {'OK':>8}\n")
```

**4. Leer todo el contenido del fichero con read().**

```
print("\n=== 4. read(): leer todo el fichero ===")
with open(ruta_servicios, "r", encoding="utf-8") as fichero:
    contenido = fichero.read()
print(contenido.strip())
```

**5. Leer líneas concretas con readline().**

```
print("\n=== 5. readline(): leer línea a línea manualmente ===")
with open(ruta_servicios, "r", encoding="utf-8") as fichero:
    cabecera = fichero.readline()
    primer_servicio = fichero.readline()
print("Cabecera:", cabecera.strip())
print("Primera línea:", primer_servicio.strip())
```

**6. Leer el fichero completo como una lista de líneas con readlines().**

```
print("\n=== 6. readlines(): leer como lista de líneas ===")
with open(ruta_servicios, "r", encoding="utf-8") as fichero:
    lineas = fichero.readlines()
print("Total de líneas:", len(lineas))
print("Línea 2:", lineas[2].strip())
```

**7. Recorrer el fichero, línea a línea con un bucle for.**

```
print("\n=== 7. Recorrido línea a línea ===")
with open(ruta_servicios, "r", encoding="utf-8") as fichero:
    for linea in fichero:
        print(linea.strip())
```

**8. Procesar líneas de texto separando columnas con split().**

```
print("\n=== 8. Procesar líneas con split() ===")
with open(ruta_servicios, "r", encoding="utf-8") as fichero:
    _ = fichero.readline()
    for linea in fichero:
        servicio, puerto, estado = linea.split()
        print(f"{'servicio'}({'puerto'}) -> Estado: {'estado'}")
```

**9. Copiar un fichero de texto leyendo de una ruta y escribiendo en otra.**

```
print("\n=== 9. Copiar fichero de texto ===")
destino = carpeta / "servicios_texto_copia.txt"
with open(ruta_servicios, "r", encoding="utf-8") as entrada, open(destino, "w", encoding="utf-8") as salida:
    for linea in entrada:
        salida.write(linea)
print("Copia creada:", destino)
```

**10. Reposicionar el cursor con seek(0) para volver a leer desde el principio.**

```
print("\n=== 10. Reposicionar cursor con seek(0) ===")
with open(ruta_servicios, "r", encoding="utf-8") as fichero:
    primera_lectura = fichero.read()
    print("Primera lectura:")
    print(primer_lectura.strip())
    _ = fichero.seek(0)
    segunda_lectura = fichero.read()
    print("Segunda lectura:")
    print(segunda_lectura.strip())
```

**11. Ejecutar el script completo desde la terminal.**

```
python Tema08/src/02_ficheros_texto.py
```

## Laboratorio 3. Operaciones con CSV

**1. Comprobar que existe el fichero Tema08/data/servicios3.csv con datos que requieren normalización.**

```
servicio,puerto,estado
SSH , 22 , OK
NGINX , 443 , ok
api , 8080 , ERROR
BACKUP ,no_numero, ERROR
dns , 53 , OK
postgresql ,5432, ok
```

**2. Abrir el fichero Tema08/src/03\_csv\_operaciones.py. Escribir un CSV básico con csv.writer.**

```
import csv
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(parents=True, exist_ok=True)

print("=== 1. Escribir CSV con csv.writer ===")
ruta_csv1 = carpeta / "servicios1.csv"
with open(ruta_csv1, "w", newline="", encoding="utf-8") as filecsv:
    escritor = csv.writer(filecsv, delimiter=",")
    escritor.writerow(["servicio", "puerto", "estado"])
    escritor.writerow(["ssh", 22, "OK"])
    escritor.writerow(["http", 80, "OK"])
    escritor.writerow(["https", 443, "OK"])
print("CSV creado:", ruta_csv1)
```

**3. Escribir un CSV estructurado con csv.DictWriter y una lista de diccionarios.**

```
print("\n=== 2. Escribir CSV con csv.DictWriter ===")
ruta_csv2 = carpeta / "servicios2.csv"
campos = ["servicio", "puerto", "estado"]
filas = [
    {"servicio": "ssh", "puerto": 22, "estado": "OK"},
    {"servicio": "http", "puerto": 80, "estado": "OK"},
    {"servicio": "postgresql", "puerto": 5432, "estado": "OK"},
]
with open(ruta_csv2, "w", newline="", encoding="utf-8") as filecsv:
    escritor = csv.DictWriter(filecsv, fieldnames=campos, delimiter=",")
    escritor.writeheader()
    escritor.writerows(filas)
print("CSV creado:", ruta_csv2)
```

**4. Leer un CSV con csv.reader y separar cabecera y filas.**

```
print("\n=== 3. Leer CSV con csv.reader ===")
with open(ruta_csv1, "r", newline="", encoding="utf-8") as filecsv:
    lector = csv.reader(filecsv, delimiter=",")
    cabecera = next(lector)
    print("Cabecera:", cabecera)
    for fila in lector:
        servicio, puerto, estado = fila
        print(f"Servicio: {servicio} -> {puerto}: {estado}")
```

**5. Leer un CSV con csv.DictReader y convertir el puerto a entero.**

```
print("\n=== 4. Leer CSV con csv.DictReader ===")
with open(ruta_csv2, "r", newline="", encoding="utf-8") as filecsv:
    lector = csv.DictReader(filecsv, delimiter=",")
    for fila in lector:
        puerto = int(fila["puerto"])
        print(f"Servicio: {fila['servicio']} -> Puerto {puerto} [{fila['estado']}]")
```

**6. Revisar el fichero Tema08/data/servicios3.csv y normalizar sus registros.**

```
print("\n=== 5. Normalizar datos desde servicios3.csv ===")
ruta_csv3 = carpeta / "servicios3.csv"
servicios_validos = []
servicios_rechazados = []

try:
    with open(ruta_csv3, "r", newline="", encoding="utf-8") as filecsv:
        lector = csv.DictReader(filecsv, delimiter=",")
        for fila in lector:
            servicio = fila["servicio"].strip().lower()
            puerto_txt = fila["puerto"].strip()
            estado = fila["estado"].strip().upper()
            try:
                puerto = int(puerto_txt)
            except ValueError:
                servicios_rechazados.append({"servicio": servicio, "puerto": puerto_txt, "motivo": "puerto no numérico"})
            else:
                servicios_validos.append({"servicio": servicio, "puerto": puerto, "estado": estado})
except FileNotFoundError:
    print(f"[CRÍTICO] No existe el fichero esperado: {ruta_csv3}")

print("Servicios válidos normalizados:")
for servicio in servicios_validos:
    print(servicio)
print("Servicios rechazados:")
for servicio in servicios_rechazados:
    print(servicio)
```

**7. Guardar los registros válidos en un nuevo CSV normalizado.**

```
print("\n=== 6. Guardar CSV normalizado ===")
ruta_normalizada = carpeta / "servicios3_normalizado.csv"
```

```
with open(ruta_normalizada, "w", newline="", encoding="utf-8") as filecsv:
    campos = ["servicio", "puerto", "estado"]
    escritor = csv.DictWriter(filecsv, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(servicios_validos)
print("CSV normalizado guardado en:", ruta_normalizada)
```

## 8. Ejecutar el script completo desde la terminal.

```
python Tema08/src/03_csv_operaciones.py
```

## Laboratorio 4. Operaciones con JSON

### 1. Abrir el fichero Tema08/src/04\_json\_operaciones.py. Crear una estructura de datos de Python basada en listas y diccionarios.

```
import json
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(parents=True, exist_ok=True)
ruta_json = carpeta / "servicios1.json"

print("=== 1. Crear datos estructurados en Python ===")
servicios = [
    {"servicio": "ssh", "puerto": 22, "estado": "OK"},
    {"servicio": "http", "puerto": 80, "estado": "OK"},
    {"servicio": "https", "puerto": 443, "estado": "ERROR"},
]
print(servicios)
```

### 2. Escribir la estructura en un fichero JSON usando `json.dump()`.

```
print("\n=== 2. Escribir JSON con json.dump() ===")
try:
    with open(ruta_json, "w", encoding="utf-8") as filejson:
        json.dump(servicios, filejson, indent=4, ensure_ascii=False)
except OSError as error:
    print("[ERROR] No se pudo escribir el archivo:", error)
else:
    print("Fichero guardado en:", ruta_json)
```

### 3. Leer el JSON con `json.load()` y recorrer los datos recuperados.

```
print("\n=== 3. Leer JSON con json.load() ===")
try:
    with open(ruta_json, "r", encoding="utf-8") as filejson:
        datos = json.load(filejson)
        for fila in datos:
            print(f"Servicio: {fila['servicio']} -> Puerto: {fila['puerto']} [{fila['estado']}]")
except FileNotFoundError:
    print(f"[CRÍTICO] El archivo no existe: {ruta_json}")
except json.JSONDecodeError:
    print("[CRÍTICO] El archivo JSON está corrupto o tiene errores.")
```

#### 4. Crear un JSON de configuración con datos anidados.

```
print("\n=== 4. Crear JSON de configuración ===")
config = {
    "aplicacion": "monitor-servicios",
    "entorno": "laboratorio",
    "reintentos": 3,
    "alertas": {"email": True, "canal": "soporte"},
}
ruta_config = carpeta / "configuracion.json"
with open(ruta_config, "w", encoding="utf-8") as filejson:
    json.dump(config, filejson, indent=4, ensure_ascii=False)
print("Configuración guardada:", ruta_config)
```

#### 5. Leer la configuración y utilizar valores concretos del diccionario recuperado.

```
print("\n=== 5. Leer configuración y usar valores ===")
with open(ruta_config, "r", encoding="utf-8") as filejson:
    config_leida = json.load(filejson)
print("Aplicación:", config_leida["aplicacion"])
print("Canal de alertas:", config_leida["alertas"]["canal"])
```

#### 6. Ejecutar el script completo desde la terminal.

```
python 04_json_operaciones.py
```

## Laboratorio 5. Serialización binaria con pickle

1. Abrir el fichero Tema08/src/05\_pickle\_operaciones.py. Crear un objeto Python que se va a serializar.

```
import pickle
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(parents=True, exist_ok=True)
ruta_pickle = carpeta / "servicios1.pkl"

print("=== 1. Crear objeto Python ===")
servicios = [
    {"servicio": "ssh", "puerto": 22, "estado": "OK"},
    {"servicio": "http", "puerto": 80, "estado": "OK"},
    {"servicio": "postgresql", "puerto": 5432, "estado": "OK"},
]
print(servicios)
```

#### 2. Serializar el objeto con pickle.dump() en un fichero binario.

```
print("\n=== 2. Serializar con pickle.dump() ===")
with open(ruta_pickle, "wb") as filepickle:
    pickle.dump(servicios, filepickle)
print("Objeto serializado en:", ruta_pickle)
```

**3. Deserializar el fichero con `pickle.load()` y recorrer los datos recuperados.**

```
print("\n=== 3. Deserializar con pickle.load() ===")
with open(ruta_pickle, "rb") as filepickle:
    datos = pickle.load(filepickle)
for fila in datos:
    print(f"Servicio: {fila['servicio']} -> {fila['puerto']} [{fila['estado']}]")
```

**4. Comprobar que `pickle` recupera tipos Python, no solo cadenas de texto.**

```
print("\n=== 4. Comprobar que se recuperan tipos Python ===")
print("Tipo del objeto recuperado:", type(datos))
print("Tipo del puerto recuperado:", type(datos[0]["puerto"]))
```

**5. Serializar y recuperar un diccionario de estado de ejecución.**

```
print("\n=== 5. Ejemplo adicional: serializar un diccionario de estado ===")
estado_ejecucion = {"ultima_ejecucion": "2026-05-19", "procesados": 3, "errores": []}
ruta_estado = carpeta / "estado_ejecucion.pkl"
with open(ruta_estado, "wb") as filepickle:
    pickle.dump(estado_ejecucion, filepickle)
with open(ruta_estado, "rb") as filepickle:
    estado_recuperado = pickle.load(filepickle)
print("Estado recuperado:", estado_recuperado)
```

**6. Ejecutar el script completo desde la terminal.**

```
python 05_pickle_operaciones.py
```

## Laboratorio 6. Ficheros binarios e imagen JPG

**1. Comprobar que existe el fichero `Tema08/data/gatito.jpg`. Se utilizará como entrada para lectura y escritura binaria.**

```
ls -lh Tema08/data/gatito.jpg
```

**2. Abrir el fichero `Tema08/src/06_binarios_imagen.py`. Comprobar que existe la imagen binaria de entrada `gatito.jpg`.**

```
from pathlib import Path

carpeta = Path("Tema08/data")
imagen_original = carpeta / "gatito.jpg"
imagen_modificada = carpeta / "gatito2.jpg"
marcador = "##INICIO##".encode("utf-8")
mensaje = "Marca de seguridad\nFichero procesado desde Python\n"

print("=== 1. Comprobar fichero binario de entrada ===")
if not imagen_original.exists():
    raise FileNotFoundError(f"No existe {imagen_original}. Copia gatito.jpg en la carpeta data.")
print("Imagen original:", imagen_original)
print("Tamaño original:", imagen_original.stat().st_size, "bytes")
```

**3. Leer los bytes de la imagen original en modo rb.**

```
print("\n=== 2. Leer bytes de la imagen original ===")
with open(imagen_original, "rb") as fichero:
    bytes_imagen = fichero.read()
print("Bytes leídos:", len(bytes_imagen))
print("Primeros 10 bytes:", bytes_imagen[:10])
```

**4. Insertar un mensaje codificado al final de una copia de la imagen.**

```
print("\n=== 3. Insertar mensaje binario al final ===")
bytes_mensaje = mensaje.encode("utf-8")
with open(imagen_modificada, "wb") as fichero:
    fichero.write(bytes_imagen + marcador + bytes_mensaje)
print("Imagen modificada:", imagen_modificada)
print("Tamaño modificado:", imagen_modificada.stat().st_size, "bytes")
```

**5. Recuperar el mensaje usando el marcador binario definido.**

```
print("\n=== 4. Recuperar mensaje usando el marcador ===")
with open(imagen_modificada, "rb") as fichero:
    contenido_binario = fichero.read()
if marcador in contenido_binario:
    partes = contenido_binario.split(marcador, maxsplit=1)
    bytes_mensaje_puros = partes[1]
    mensaje_extraido = bytes_mensaje_puros.decode("utf-8")
    print("--- Mensaje oculto recuperado ---")
    print(mensaje_extraido)
else:
    print("No se encontró el marcador en el fichero.")
```

**6. Crear una copia binaria simple por bloques.**

```
print("\n=== 5. Copia binaria simple por bloques ===")
copia = carpeta / "gatito_copia.jpg"
with open(imagen_original, "rb") as entrada, open(copia, "wb") as salida:
    while True:
        bloque = entrada.read(1024)
        if not bloque:
            break
        salida.write(bloque)
print("Copia creada:", copia)
print("Tamaño copia:", copia.stat().st_size, "bytes")
```

**7. Ejecutar el script completo desde la terminal.**

```
python Tema08/src/06_binarios_imagen.py
```

## Laboratorio 7. Ejercicio integrado: normalizar CSV y exportar a JSON

### 1. Comprobar que existe el fichero Tema08/data/inventario\_empresa.csv. Este será el CSV empresarial de entrada.

```
departamento,servidor,servicio,puerto,estado,criticidad
Sistemas,srv-web-01, NGINX ,443,activo,alta
```

```

Sistemas,srv-web-01,ssh,22,activo,media
Datos,srv-db-01,POSTGRESQL,5432,activo,alta
Soporte,srv-help-01,api,8080,detenido,media
Sistemas,srv-bkp-01,backup,70000,activo,alta
Redes,srv-dns-01,dns,53,activo,media
Redes,srv-vpn-01,vpn,no_numero,activo,alta

```

**2. Abrir el fichero** Tema08/src/07\_ejercicio\_integrado\_csv\_a\_json.py. **Comprobar que existe el CSV de entrada** inventario\_empresa.csv.

```

import csv
import json
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(parents=True, exist_ok=True)
entrada_csv = carpeta / "inventario_empresa.csv"
salida_json = carpeta / "inventario_empresa_normalizado.json"
salida_rechazados = carpeta / "inventario_empresa_rechazados.csv"

print("=== 1. Comprobar fichero de entrada ===")
if not entrada_csv.exists():
    raise FileNotFoundError(f"No existe el fichero de entrada: {entrada_csv}")
print("Entrada:", entrada_csv)

```

**3. Definir funciones auxiliares para normalizar texto, validar puertos y normalizar filas.**

```

print("\n=== 2. Funciones auxiliares ===")
def normalizar_texto(valor):
    """Limpia espacios y convierte texto a minúsculas."""
    return valor.strip().lower()

def validar_puerto(puerto_txt):
    """Convierte y valida un puerto."""
    puerto = int(puerto_txt)
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return puerto

def normalizar_fila(fila):
    """Normaliza una fila del CSV de inventario."""
    return {
        "departamento": normalizar_texto(fila["departamento"]),
        "servidor": normalizar_texto(fila["servidor"]),
        "servicio": normalizar_texto(fila["servicio"]),
        "puerto": validar_puerto(fila["puerto"].strip()),
        "estado": normalizar_texto(fila["estado"]),
        "criticidad": normalizar_texto(fila["criticidad"]),
    }

print("Funciones definidas.")

```

**4. Leer el CSV, normalizar registros válidos y separar registros rechazados.**

```
print("\n=== 3. Leer, normalizar y validar CSV ===")
registros_validos = []
registros_rechazados = []
with open(entrada_csv, "r", newline="", encoding="utf-8") as filecsv:
    lector = csv.DictReader(filecsv)
    for numero_linea, fila in enumerate(lector, start=2):
        try:
            normalizada = normalizar_fila(fila)
        except ValueError as error:
            rechazada = dict(fila)
            rechazada["linea"] = numero_linea
            rechazada["motivo"] = str(error)
            registros_rechazados.append(rechazada)
        else:
            registros_validos.append(normalizada)

print("Registros válidos:", len(registros_validos))
print("Registros rechazados:", len(registros_rechazados))
```

**5. Construir la estructura final que se exportará como JSON.**

```
print("\n=== 4. Construir estructura JSON final ===")
salida = {
    "origen": str(entrada_csv),
    "total_validos": len(registros_validos),
    "total_rechazados": len(registros_rechazados),
    "servicios": registros_validos,
}
print(json.dumps(salida, indent=4, ensure_ascii=False))
```

**6. Escribir el JSON normalizado en disco.**

```
print("\n=== 5. Escribir JSON normalizado ===")
with open(salida_json, "w", encoding="utf-8") as filejson:
    json.dump(salida, filejson, indent=4, ensure_ascii=False)
print("JSON generado:", salida_json)
```

**7. Escribir un CSV con los registros rechazados y el motivo del rechazo.**

```
print("\n=== 6. Escribir CSV de rechazados ===")
campos_rechazados = ["linea", "departamento", "servidor", "servicio", "puerto",
"estado", "criticidad", "motivo"]
with open(salida_rechazados, "w", newline="", encoding="utf-8") as filecsv:
    escritor = csv.DictWriter(filecsv, fieldnames=campos_rechazados)
    escritor.writeheader()
    escritor.writerows(registros_rechazados)
print("CSV de rechazados generado:", salida_rechazados)
```

**8. Leer el JSON generado para comprobar el resultado final.**

```
print("\n=== 7. Leer el JSON generado para comprobarlo ===")
with open(salida_json, "r", encoding="utf-8") as filejson:
    comprobacion = json.load(filejson)

for servicio in comprobacion["servicios"]:
    print(f"{servicio['servidor']} -> {servicio['servicio']}: {servicio['puerto']} [{servicio['estado']}]")
```

## 9. Ejecutar el script completo desde la terminal.

```
python 07_ejercicio_integrado_csv_a_json.py
```

## Ejercicios propuestos

1. Crear un script `src/reporte_texto_servicios.py` que genere un fichero `reporte_servicios.txt` con una tabla de servicios y puertos. Después debe leerlo y mostrarlo por pantalla.
2. Crear un script `src/anadir_log.py` que escriba varias líneas de log en modo a sobre `Tema08/data/eventos.log` sin borrar las anteriores.
3. Crear un script `src/filtrar_csv_activos.py` que lea un CSV de servicios y genere otro CSV solo con los registros cuyo estado sea activo u OK.
4. Crear un script `src/config_json.py` que cree un fichero `configuracion_app.json` con parámetros de una aplicación y luego lo vuelva a leer para mostrar algunos valores.
5. Crear un script `src/backup_pickle.py` que guarde con `pickle` una lista de tareas de backup y la recupere mostrando cuántas tareas hay pendientes.
6. Crear un script `src/copiar_binario.py` que copie un fichero binario por bloques de 512 bytes y compare el tamaño del original y la copia.
7. Crear un script `src/convertir_empleados_csv_json.py` que lea un CSV pequeño de empleados, normalice nombres y departamentos, y exporte el resultado a JSON.

## Guía de resolución de problemas

| Problema                                                            | Diagnóstico                                                                       | Solución                                                                                                          |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| No aparece (Curso_Python) en el prompt                              | El entorno virtual no está activado.                                              | Ejecutar <code>source .venv/bin/activate</code> desde <code>~/Curso_Python/Tema08</code> .                        |
| El script no encuentra la carpeta <code>data</code>                 | El script se ejecuta desde un directorio distinto al esperado.                    | Ejecutar <code>cd ~/Curso_Python/Tema08</code> antes de lanzar el script o revisar las rutas relativas.           |
| <code>FileNotFoundError</code> al abrir <code>servicios3.csv</code> | El fichero no está en <code>Tema08/data</code> o el nombre no coincide.           | Copiar <code>servicios3.csv</code> en <code>Tema08/data</code> y comprobar <code>ls -la Tema08/data</code> .      |
| <code>FileNotFoundError</code> al abrir <code>gatito.jpg</code>     | La imagen binaria de entrada no está copiada en el directorio <code>data</code> . | Copiar <code>gatito.jpg</code> en <code>Tema08/data</code> antes de ejecutar <code>06_binarios_imagen.py</code> . |
| <code>UnicodeDecodeError</code> al leer un fichero de texto         | El fichero no está codificado como UTF-8.                                         | Comprobar la codificación real del fichero o abrirlo con la codificación adecuada.                                |

| Problema                                              | Diagnóstico                                                                              | Solución                                                                                                        |
|-------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| El CSV genera líneas en blanco en algunos sistemas    | No se ha usado <code>newline=""</code> al abrir el fichero CSV.                          | Abrir los CSV con <code>open(ruta, "w", newline="", encoding="utf-8")</code> .                                  |
| <code>ValueError</code> al convertir un puerto        | El campo del CSV contiene texto o un número fuera de rango.                              | Capturar <code>ValueError</code> y registrar el dato como rechazado.                                            |
| <code>JSONDecodeError</code> al leer JSON             | El fichero JSON está vacío, truncado o tiene sintaxis inválida.                          | Regenerar el JSON con <code>json.dump()</code> y validar que no se editó manualmente con errores.               |
| <code>pickle.load()</code> falla o resulta sospechoso | El fichero <code>pickle</code> puede estar dañado o proceder de una fuente no confiable. | No cargar <code>pickle</code> de origen desconocido. Regenerarlo desde datos confiables.                        |
| La copia binaria no abre correctamente                | Se ha usado modo texto en vez de modo binario o no se copió todo el contenido.           | Usar <code>rb</code> y <code>wb</code> , y copiar en bucle hasta que <code>read()</code> devuelva bytes vacíos. |